Pneumonia Detection from Chest X-rays

How the project was built — a plain-language walkthrough

---

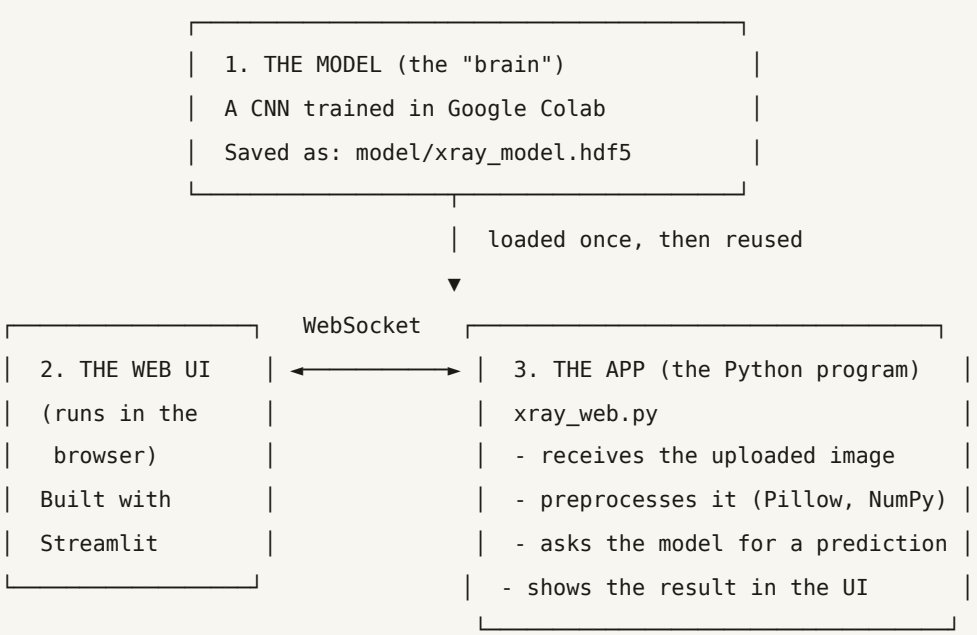
1. What the project does, in one paragraph

The project is a web application that looks at a chest X-ray image and decides whether it shows **signs of pneumonia** or looks **normal**. A user opens the page in a browser, uploads an X-ray picture, and the app returns a label (`NORMAL` or `PNEUMONIA`) together with a confidence percentage. The decision is made by a **machine-learning model** — a Convolutional Neural Network (CNN) that was trained on thousands of labelled X-ray images in advance.

---

2. The big picture — three parts

The system is made of three pieces that work together:



- **The model** was created *offline* (in a Google Colab notebook) and saved to a file.
- **The app** is a single Python script that loads that file and serves a web page.
- **The web UI** is generated automatically by Streamlit and rendered in the user's browser.

There is **no separate database**, just the saved model file and a small text file listing the two class names.

---

3. The AI model — what type it is and how it is built

What kind of model is it?

It is a **Convolutional Neural Network (CNN)**. A CNN is the standard type of neural network for images. Instead of looking at every pixel individually, it slides small filters ("kernels") across the image to detect visual patterns — edges, curves, and then larger shapes. Early layers detect simple things; deeper layers combine them into more complex features.

> Note: a CNN has no idea what an "X-ray" is in words. It just learns which patterns of pixels tend to appear in each class.

The exact architecture (from the saved model file)

It is a small, **sequential** CNN, built layer by layer:

Step	Layer	What it does	Output size
0	Input	Receives the image	180 × 180 × 3
1	Rescaling (÷255)	Turns pixel values from 0–255 into 0–1	180 × 180 × 3
2	Conv2D (16 filters) + MaxPool	Finds 16 simple features, then halves the size	90 × 90 × 16
3	Conv2D (32 filters) + MaxPool	Finds 32 richer features	45 × 45 × 32
4	Conv2D (64 filters) + MaxPool	Finds 64 even richer features	22 × 22 × 64
5	Flatten	Turns the grid into one long list of numbers	30,976
6	Dense (128, ReLU)	Learns combinations of features	128
7	Dense (2, Softmax)	Produces a probability for each class	2

- **Total learnable parameters:** ≈ 3.99 million (about 15 MB).
- **Input:** a 180 × 180 pixel RGB image.
- **Output:** two numbers that add up to 1 — e.g. `[0.01, 0.99]` means "1 % normal, 99 % pneumonia".

A few terms in plain words:

- **Filter / kernel:** a small window the network slides over the image to detect a pattern.
- **ReLU:** a simple rule that keeps positive numbers and turns negatives into zero; it lets the network learn curved, non-linear relationships.
- **MaxPool:** shrinking the image by keeping only the strongest signal in each small region (this makes the model faster and more robust).
- **Dense layer:** a normal fully-connected layer of neurons.
- **Softmax:** converts the network's raw scores into probabilities that sum to 1.

- **Parameter:** a number the network adjusts during training. "Learning" is just finding good values for these numbers.

A note on the README

The project README mentions "ResNet architectures" and "transfer learning". To be precise for a technical audience: **the model that was actually trained and shipped is a small CNN built from scratch**, not a pretrained ResNet. Transfer learning would mean taking a model already trained on millions of general images (e.g. ResNet-50 on ImageNet) and fine-tuning it. That approach is common and often gives better accuracy, but it is **not** what this particular model is. When explaining the project, describe it as "a custom CNN trained from scratch", which matches the model file.

---

4. The training data

- **Name:** *Chest X-Ray Images (Pneumonia)*
- **Where it comes from:** a public dataset on Kaggle (published by Paul Mooney), originally collected at the **Guangzhou Women and Children's Medical Center** in Guangzhou, China.
- **What it contains:** ~5,860 chest X-ray images (JPEG), all taken from the front of the chest (anterior–posterior), of **children aged 1 to 5 years old**.
- **Classes (labels):** two categories —
  - `NORMAL` — healthy lungs
  - `PNEUMONIA` — lungs showing pneumonia
- **Quality control:** before release, low-quality scans were removed, and each image was graded by expert physicians, with a third expert double-checking the evaluation set.

How the data was split in this project

Split	Images	Used for
Train	5,242	Teaching the model
Test	624	Checking the model during training (validation) and afterwards

> Note: the dataset also contains a tiny official "val" folder (only a handful of images). This project used the **Test** folder for validation instead, because the official val set is too small to be meaningful.

The raw dataset is **not stored in this repository** (it is large). Only a few example images are included under `sample images/` for testing.

---

5. How the model was trained

The training happened **offline in a notebook** (`notebooks/chest_xray.ipynb`), not in the web app. Here is exactly what was done:

Where and with what

- Platform: **Google Colab** with a free GPU (training on CPU would be far slower).
- Framework: **TensorFlow 2 / Keras** (the standard Python deep-learning library).
- Data loaded with `tf.keras.preprocessing.image_dataset_from_directory` , which reads folders named by class and assigns labels automatically.

Settings (hyper-parameters)

Setting	Value	Meaning in plain words
Image size	180 × 180	Every image was resized to this before training
Batch size	32	The model looked at 32 images at a time
Epochs	3	It went through the whole training set 3 times
Optimizer	Adam	The algorithm that adjusts the parameters
Loss function	Sparse categorical cross-entropy	Measures how wrong the model is; training tries to reduce it
Random seed	123	Makes the shuffling reproducible

The training loop, in plain words:

1. Take a batch of 32 images and their true labels.
2. Let the model guess each one.
3. Compare the guesses to the truth and measure the error (the "loss").
4. Slightly adjust all ~4 million parameters to reduce that error.
5. Repeat for all batches, three times over.

What happened during training (recorded results):

Epoch	Training accuracy	Validation accuracy
1	93.1 %	69.2 %
2	96.5 %	78.2 %
3	97.6 %	81.3 %

After training, the model was saved to a file with `tf.keras.models.save_model(model, 'xray_model.hdf5')` — that file is `model/xray_model.hdf5` in this repository.

How to read these numbers: training accuracy kept rising, but validation accuracy was much lower and improved more slowly. That gap is a classic sign of **overfitting** — the model started memorising the training

images instead of learning general patterns. Training longer with a proper validation strategy (and more epochs) would be the normal next step to improve it.

One technical detail worth knowing: the notebook used the loss function with the flag `from_logits=True` even though the final layer already applies `softmax`. These two settings are slightly inconsistent (TensorFlow prints a warning about it). Training still worked and the model learned, but the correct fix is either to remove the softmax from the last layer (using pure logits) or to set `from_logits=False`.

---

6. The application — frontend and backend

Frontend (what the user sees)

- Built with **Streamlit**, a Python framework that turns a Python script into an interactive web page with almost no HTML or JavaScript required. It handles the file upload, the layout, and the live updates automatically.
- The look and feel (dark "reading-room" theme, amber accent, custom fonts) is layered on top with **custom CSS** and two **Google Fonts** (Fraunces for headings, Instrument Sans for body text).
- The page shows: a title, an upload area, the X-ray image, and a "verdict" card with the predicted class, a confidence percentage, and probability bars.

Backend (what happens behind the scenes)

There is **no separate backend server or REST API**. Streamlit itself runs a small Python web server, and the same script is both the "frontend" and the "backend". The backend logic is the **inference pipeline** in `xray_web.py`:

1. The uploaded image file arrives in the Python program.
2. **Pillow** opens it and resizes/crops it to exactly 180 × 180 pixels in RGB.
3. **NumPy** converts it into the numeric array the model expects.
4. **TensorFlow / Keras** runs `model.predict(...)` and returns the two probabilities.
5. The program picks the higher one (`argmax`) and sends the result to the UI.

How Streamlit actually runs (useful to mention): Streamlit keeps a live connection to the browser over a **WebSocket**. Every time the user interacts (e.g. uploads a file), the script re-runs from top to bottom. To avoid re-loading the model every time (which is slow), the model is cached with `@st.cache_resource` and the class names with `@st.cache_data`. The model is therefore loaded only once per session and reused.

The preprocessing detail that matters

The model already contains a **Rescaling** layer that divides pixel values by 255. So the app must feed it pixels in the **0–255** range, *not* pre-divided — otherwise the scaling would be applied twice. The app correctly keeps the raw 0–255 values and lets the model do the scaling internally.

7. Frameworks and tools used (summary table)

Part	Tool / framework	Role
Web interface	**Streamlit**	Builds the web page and handles user interaction
Styling	**Custom CSS + Google Fonts**	Dark theme, typography, layout polish
Language	**Python 3.13**	Everything is written in Python
Deep learning	**TensorFlow 2 / Keras**	Defines, trains, saves, and runs the CNN
Image handling	**Pillow (PIL)**	Opens and resizes uploaded images
Numbers / arrays	**NumPy**	Converts images to arrays for the model
Training platform	**Google Colab (GPU)**	Where the model was trained
Data source	**Kaggle Chest X-Ray dataset**	The labelled X-ray images
Model file format	**HDF5 (`.hdf5`)**	The saved trained model

> The `requirements.txt` file also lists `opencv-python`, which the original prototype used. The current app does **not** need OpenCV — it uses Pillow instead. If installing fresh, the needed packages are `streamlit`, `tensorflow`, `numpy`, and `pillow`.

8. A single prediction, step by step

1. **User** selects a chest X-ray in the browser and it is uploaded.
2. **Streamlit** passes the file to the Python script.
3. The script **opens** the image, **resizes** it to 180 × 180, and **converts** it to a numeric array.
4. The array is passed to the **CNN**, which outputs two probabilities.
5. The script takes the **larger probability** as the answer.
6. The **UI** displays the class, the confidence (e.g. "Pneumonia — 74.3 %"), and a bar for each class.

The whole thing takes a fraction of a second on a normal laptop, because only *inference* happens here — the heavy *training* already happened earlier in Colab.

9. Results and honest limitations

Strengths

- Complete end-to-end system: data → trained model → working web app.
- The model reaches ~97 % accuracy on the data it trained on and ~81 % on unseen data.
- The app is fast, works in any browser, and requires no GPU at prediction time.

Limitations (worth stating openly, professors appreciate this)

1. **Overfitting** — the gap between training and validation accuracy shows the model memorised some training data.
2. **Only 3 epochs** — far more training would likely improve accuracy.
3. **Small, specific dataset** — the images are of children aged 1–5 only, so the model may not generalise to adults or to different hospitals/scanners.
4. **It is not a medical device** — it is an educational prototype. Any real diagnostic use would require clinical validation, much more data, and regulatory approval.
5. **README vs. reality** — the README mentions ResNet/transfer learning, but the shipped model is a simple CNN trained from scratch.

Possible future improvements

- Train for more epochs and use data augmentation (rotations, flips, brightness changes) and early stopping.
- Add a proper validation set and track metrics such as precision, recall, and a confusion matrix.
- Use **transfer learning** with a pretrained network (e.g. ResNet or EfficientNet) and fine-tune it — this usually gives a strong accuracy boost.
- Evaluate separately on normal vs. pneumonia patients to check for bias.

---

10. How to run the project

```
# 1. Install the required packages
pip install streamlit tensorflow numpy pillow

# 2. Start the web app (run this from the project folder)
streamlit run xray_web.py

# 3. Open the address Streamlit prints (usually http://localhost:8501)
#    and upload a chest X-ray image.
```

Key files in the project

File	Purpose
`xray_web.py`	The Streamlit web application (frontend + inference logic)

File	Purpose
`model/xray_model.hdf5`	The trained CNN
`model/model_classes.txt`	The two class names (`NORMAL` , `PNEUMONIA`)
`notebooks/chest_xray.ipynb`	The notebook where the model was trained
`sample images/`	Example X-rays for testing
`.streamlit/config.toml`	Theme settings for the app
`requirements.txt`	Python dependencies

---

11. A 30-second version (for the spoken explanation)

> "I built a web app that detects pneumonia from chest X-rays. The AI is a Convolutional Neural Network — a type of deep-learning model that's standard for images — built with TensorFlow and Keras. I trained it in Google Colab on about 5,800 X-rays from a public dataset of children from a hospital in Guangzhou, split into a training set and a test set. It looks at 180-by-180-pixel images and outputs the probability of 'normal' versus 'pneumonia'. The frontend and the app logic are both built with Streamlit, a Python web framework, so the same script serves the page and runs the model. When a user uploads an X-ray, the app resizes it, feeds it to the model, and shows the predicted class with a confidence score. The model reaches about 81 % accuracy on unseen images — it's an educational prototype, not a medical device."